

Atelier Machine Learning

Analyse Comportementale Clientèle Retail

COMPTE RENDU

Atelier Pratique : E-commerce de Cadeaux

Préparé par Ala Belguith

Année Universitaire : 2025-2026

1 Introduction

Ce rapport décrit le travail réalisé dans le notebook de mon travail pour l'atelier Machine Learning portant sur l'analyse comportementale de la clientèle d'un e-commerce de cadeaux.

1.1 Objectif du notebook

Le notebook prend en entrée les **52 features** et applique le pipeline suivant :

Pipeline de traitement

- Étape 1 Exploration du Dataset
- Étape 2 Encodage + feature engineering
- Étape 3 Matrice de corrélation
- Étape 4 Suppression des features selon les 4 critères
 - Variance nulle
 - Trop de valeurs manquantes (> 50%)
 - Forte corrélation (redondance, $|r| > 0.85$)
 - Importance nulle (Random Forest)
- Étape 5 Imputation avancée (Médiane · KNN · Iterative)
- Étape 6 Normalisation (StandardScaler -- sur train uniquement)
- Étape 7 PCA → réduction vers 2-10 composantes
- Étape 8 Split Train/Test + SMOTE
- Étape 9 Export

1.2 Contexte métier

L'entreprise e-commerce souhaite prédire le **churn** client (variable binaire **Churn** : 0 = fidèle, 1 = parti) afin de personnaliser ses stratégies marketing et réduire le taux d'attrition. Le dataset contient **4 372 observations** avec un taux de churn de **33,3%**.

2 Étape 0 — Imports et Configuration

```
1 import pandas as pd
2 import numpy as np
3 from sklearn.preprocessing import StandardScaler
4 from sklearn.impute import KNNImputer, IterativeImputer
5 from sklearn.decomposition import PCA
6 from sklearn.ensemble import RandomForestClassifier
7 from sklearn.feature_selection import VarianceThreshold
8 from imblearn.over_sampling import SMOTE
9 import joblib
```

Listing 1 – Imports principaux

Bibliothèque	Usage	Module clé
pandas, numpy	Manipulation des données	—
matplotlib, seaborn	Visualisation	sns.set_theme
sklearn.preprocessing	Normalisation	StandardScaler
sklearn.impute	Imputation	KNNImputer, IterativeImputer
sklearn.decomposition	Réduction de dimension	PCA
sklearn.ensemble	Importance features	RandomForestClassifier
sklearn.feature_selection	Filtrage variance	VarianceThreshold
imblearn.over_sampling	Rééquilibrage	SMOTE
joblib	Persistance des modèles	—

3 Étape 1 — Exploration du Dataset (EDA)

Avant toute transformation, une exploration visuelle complète du dataset brut est réalisée afin de comprendre la structure des données, détecter les anomalies et guider les choix de prétraitement. Le dataset comporte **4 372 clients** et **52 colonnes** couvrant des dimensions comportementales, transactionnelles et démographiques.

3.1 Structure générale du dataset

Le dataset contient un mélange de variables numériques continues (montants, fréquences, délais) et de variables catégorielles (segments RFM, type de client, pays). Plusieurs colonnes présentent des formats de date inconsistants ou des valeurs sentinelles (-1 , 999) qui nécessitent un nettoyage ciblé.

3.2 Valeurs manquantes

Seules cinq colonnes présentent des données manquantes. Aucune ne dépasse le seuil de 50 %, ce qui exclut toute suppression de colonne pour ce motif. Les stratégies d'imputation retenues varient selon la nature de chaque variable :

- **Age** ($\approx 30\%$ de NaN) — imputation par KNN ($k = 5$) pour exploiter la similarité entre profils clients.
- **SatisfactionScore** ($\approx 8\%$ de NaN) — Iterative Imputer (régression multivariée MICE).
- **SupportTicketsCount** et **SupportBurden** ($\approx 3\%$ de NaN) — imputation par la médiane.
- **AvgDaysBetweenPurchases** ($\approx 1,8\%$ de NaN) — imputation par la médiane.

TABLE 1 – Récapitulatif des valeurs manquantes par feature

Feature	NaN (train)	% manquant
Age	1 061	30,34 %
SatisfactionScore	286	8,18 %
SupportTicketsCount	105	3,00 %
SupportBurden	105	3,00 %
AvgDaysBetweenPurchases	66	1,89 %

3.3 Distribution de la variable cible — Churn

Le taux de churn global est de **33,3 %** (1 457 clients partis sur 4 372). Ce déséquilibre de classe modéré (ratio 2 :1) ne compromet pas directement l'entraînement, mais nécessite l'application de **SMOTE** (*Synthetic Minority Over-sampling Technique*) sur l'ensemble d'entraînement pour éviter que le modèle ne soit biaisé vers la classe majoritaire.

[title=Synthèse EDA — Points clés]

- Taux de churn : **33,3 %** (déséquilibre modéré → SMOTE sur **X_train** nécessaire).
- **Recency** et **ChurnRiskCategory** : corrélations $> 0,85$ avec Churn → prédictors clés à conserver.
- **Age** : $\approx 30\%$ de valeurs manquantes → KNN Imputer ($k = 5$).
- **SatisfactionScore** : $\approx 8\%$ de NaN → Iterative Imputer.
- **NewsletterSubscribed** : variance nulle (100 % "Yes") → supprimée immédiatement.
- Nombreuses paires monétaires corrélées ($|r| > 0,85$) → suppression des redondances à l'étape 3.
- Valeurs sentinelles : -1 , 999 , 99 dans tickets/satisfaction → remplacées par NaN.

4 Étape 2 — Encodage et Feature Engineering

À partir du dataset brut exploré en Étape 1, cette étape applique l'ensemble des transformations nécessaires pour passer de **52 colonnes brutes** à **87 features numériques** prêtes pour la modélisation.

4.1 Suppression de la feature constante

La colonne `NewsletterSubscribed` prend uniquement la valeur "Yes" pour tous les clients (variance nulle, aucune information discriminante).

```
1 df.drop(columns=['NewsletterSubscribed'], inplace=True)
2 print('Dropped NewsletterSubscribed (variance = 0)')
```

4.2 Parsing de RegistrationDate

La colonne de date présentait des formats inconsistants (UK DD/MM/YY, ISO YYYY-MM-DD, US MM/DD/YYYY). Quatre features temporelles ont été extraites : `RegYear`, `RegMonth`, `RegDay`, `RegWeekday`.

```
1 df['RegistrationDate'] = pd.to_datetime(
2     df['RegistrationDate'],
3     dayfirst=True,      # priorite format UK (DD/MM)
4     errors='coerce'    # NaT si format non reconnu
5 )
6 df['RegYear']          = df['RegistrationDate'].dt.year
7 df['RegMonth']         = df['RegistrationDate'].dt.month
8 df['RegDay']           = df['RegistrationDate'].dt.day
9 df['RegWeekday']       = df['RegistrationDate'].dt.weekday
10 df.drop(columns=['RegistrationDate'], inplace=True)
```

4.3 Feature Engineering sur LastLoginIP

Deux features ont été extraites de l'adresse IP brute :

- `IP_IsPrivate` : indicateur (1 si IP RFC-1918, 0 sinon, -1 si invalide)
- `IP_FirstOctet` : premier octet, proxy de la plage réseau

4.4 Correction des valeurs aberrantes

Feature	Valeurs aberrantes	Traitement
<code>SupportTicketsCount</code>	-1 (sentinel), 999 (erreur)	Remplacées par NaN
<code>SatisfactionScore</code>	-1, 0, 99	Remplacées par NaN

4.5 Encodage ordinal (6 features)

Les variables catégorielles à ordre naturel sont encodées avec une échelle entière préservant la hiérarchie métier.

Feature	Échelle
<code>AgeCategory</code>	Inconnu(0) → 65+(6)
<code>SpendingCategory</code>	Low(1) → VIP(4)
<code>LoyaltyLevel</code>	Nouveau(1) → Ancien(4)
<code>ChurnRiskCategory</code>	Faible(1) → Critique(4)
<code>BasketSizeCategory</code>	Inconnu(0) → Grand(3)
<code>PreferredTimeOfDay</code>	Nuit(0) → Soir(4)

4.6 One-Hot Encoding (8 features → 31 colonnes binaires)

Les features nominales (RFMSegment, CustomerType, FavoriteSeason, WeekendPreference, ProductDiversity, Gender, AccountStatus, Region) ont été transformées via `get_dummies` avec `drop_first=True` pour éviter la multicolinéarité parfaite.

4.7 Target Encoding — Country

Avec plus de 37 niveaux, **Country** a été encodé par le taux de churn moyen par pays (encodage supervisé), évitant l'explosion dimensionnelle du One-Hot Encoding.

```
1 country_churn_map = df.groupby('Country')['Churn'].mean()
2 df['Country_TargetEnc'] = df['Country'].map(country_churn_map)
3 df.drop(columns=['Country'], inplace=True)
```

4.8 Feature Engineering — 10 nouvelles features dérivées

Dix features métier ont été construites à partir des variables existantes pour enrichir le pouvoir prédictif du modèle.

TABLE 2 – Features dérivées créées lors du feature engineering

Feature créée	Description / Formule
MonetaryPerDay	MonetaryTotal / (Recency+1) — Intensité de dépense
AvgBasketValue	MonetaryTotal / Frequency — Valeur du panier moyen
TenureRecencyRatio	Recency / (CustomerTenure+1) — Inactivité relative
CancelPerFreq	CancelledTrans / (Frequency+1) — Taux d'annulation
MaxOrderShare	MonetaryMax / (MonetaryTotal +1) — Part commande max
SupportBurden	SupportTickets / (CustomerTenure+1) — Charge support
ProdBreadthPerTrans	UniqueProducts / (Frequency+1) — Largeur produits
IsNewCustomer	$1[\text{Tenure} < 90]$ — Flag client récent
HighCancelFlag	$1[\text{Cancel}/\text{Total} > 0,10]$ — Fort taux annulation
RegYearsActive	$2011 - \text{RegYear}$ — Ancienneté du compte

Résultat après l'Étape 2

4 372 lignes × 87 features (86 features numériques + 1 target Churn)

5 Étape 3 — Sélection de Features

La sélection de features vise à éliminer les variables redondantes, non informatives ou parasites avant l'entraînement des modèles. Quatre critères successifs sont appliqués, réduisant l'espace de **86 à 58 features**.

5.1 Matrice de corrélation complète (86 features)

La heatmap ci-dessous présente la matrice de corrélation de Pearson entre l'ensemble des 86 features après encodage. Les valeurs manquantes sont temporairement remplacées par la médiane pour permettre le calcul. Les clusters de fortes corrélations visibles dans la partie supérieure gauche correspondent principalement aux variables monétaires et comportementales dérivées.

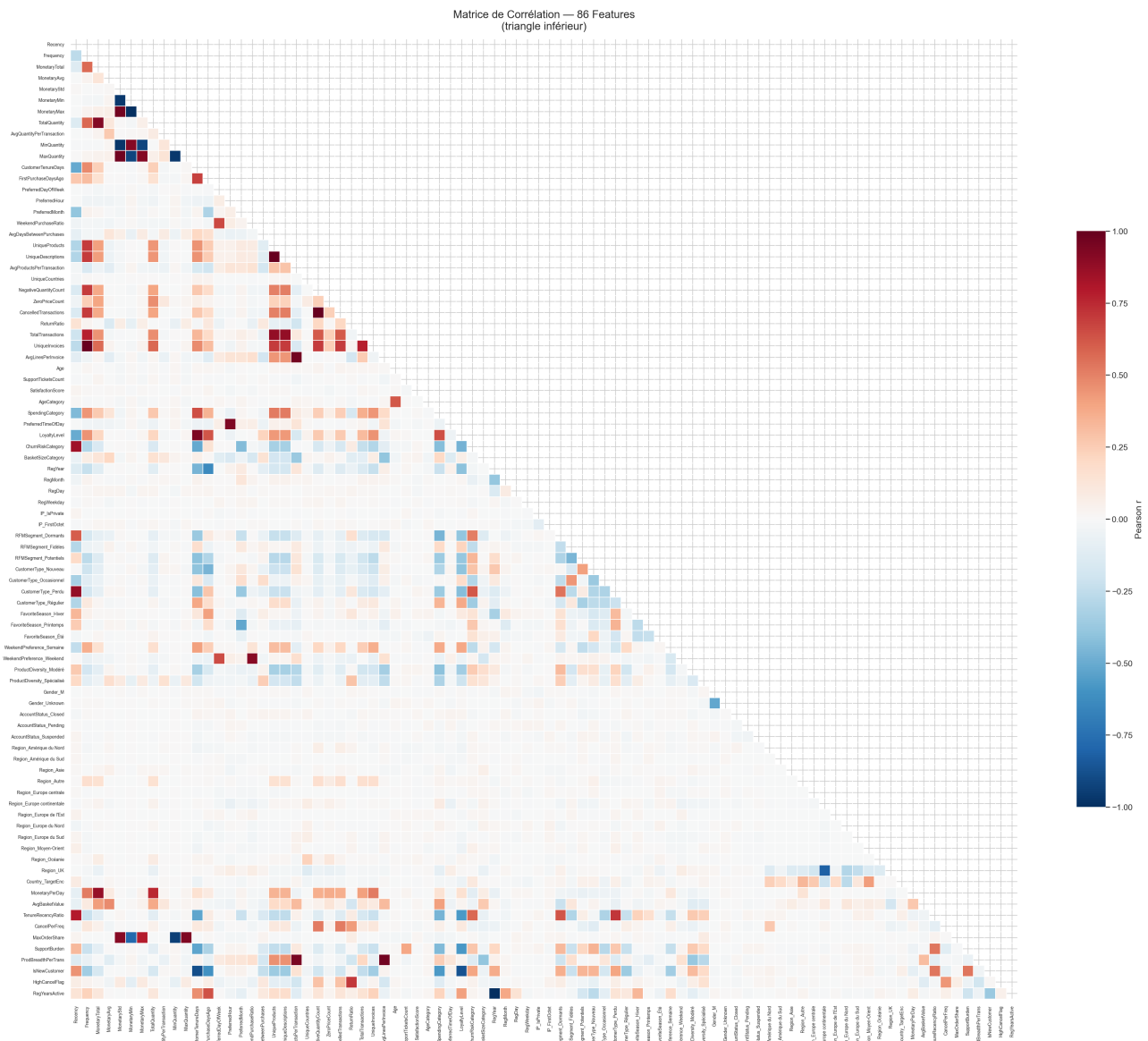


FIGURE 8 – Matrice de corrélation de Pearson sur les 86 features encodées (heatmap triangulaire inférieure).

5.2 Features les plus corrélées avec le Churn

Le barplot ci-dessous classe les 30 features selon leur corrélation absolue avec la variable cible Churn. Les deux prédicteurs dominants sont **ChurnRiskCategory** ($|r| = 0,879$) et **Recency** ($|r| = 0,859$), confirmant

intuitivement que les clients inactifs récemment sont les plus susceptibles d'avoir churné.

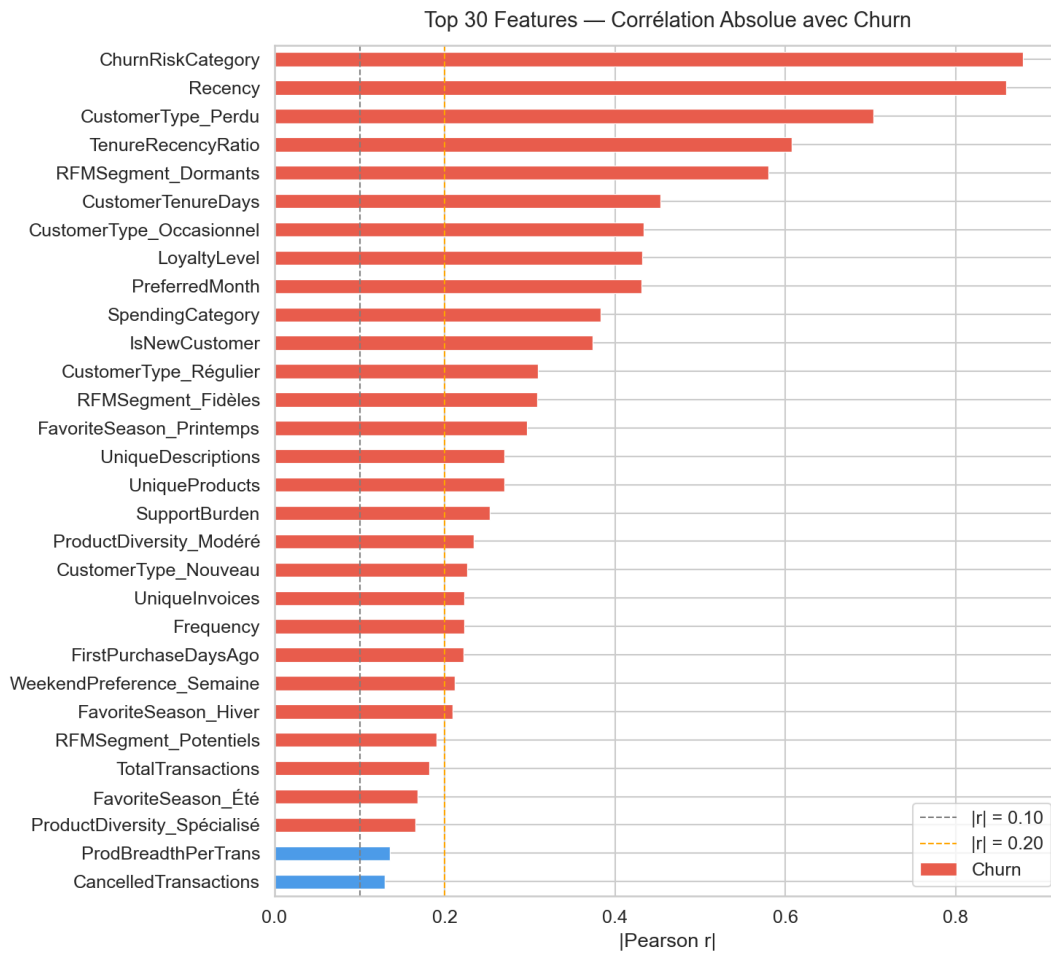


FIGURE 9 – Top 30 features classées par corrélation absolue de Pearson avec le Churn.

TABLE 3 – Top 10 features les plus corrélées avec le Churn

Rang	Feature	$ r $
1	ChurnRiskCategory	0,879
2	Recency	0,859
3	CustomerType_Perdu	0,703
4	TenureRecencyRatio	0,607
5	RFMSegment_Dormants	0,580
6	CustomerTenureDays	0,454
7	CustomerType_Occasionnel	0,434
8	LoyaltyLevel	0,431
9	PreferredMonth	0,431
10	SpendingCategory	0,383

5.3 Critère 1 — Variance nulle

Le filtre `VarianceThreshold(threshold=0.0)` détecte **0 feature à variance nulle**. La colonne `NewsletterSubscribed` avait déjà été supprimée à l'étape précédente.

```

1 vt = VarianceThreshold(threshold=0.0)
2 vt.fit(X_corr)
3 zero_var_cols = X_full.columns[~vt.get_support()].tolist()

```

4 # Features a variance nulle : 0

5.4 Critère 2 — Valeurs manquantes > 50%

Aucune feature ne dépasse le seuil de 50% de valeurs manquantes (maximum observé : Age à 29,99%). Aucune suppression à cette étape.

5.5 Critère 3 — Forte corrélation inter-features ($|r| > 0,85$)

30 paires de features présentent une corrélation supérieure au seuil de 0,85, révélant une redondance informationnelle importante. Pour chaque paire, la feature conservée a été choisie selon le sens métier.

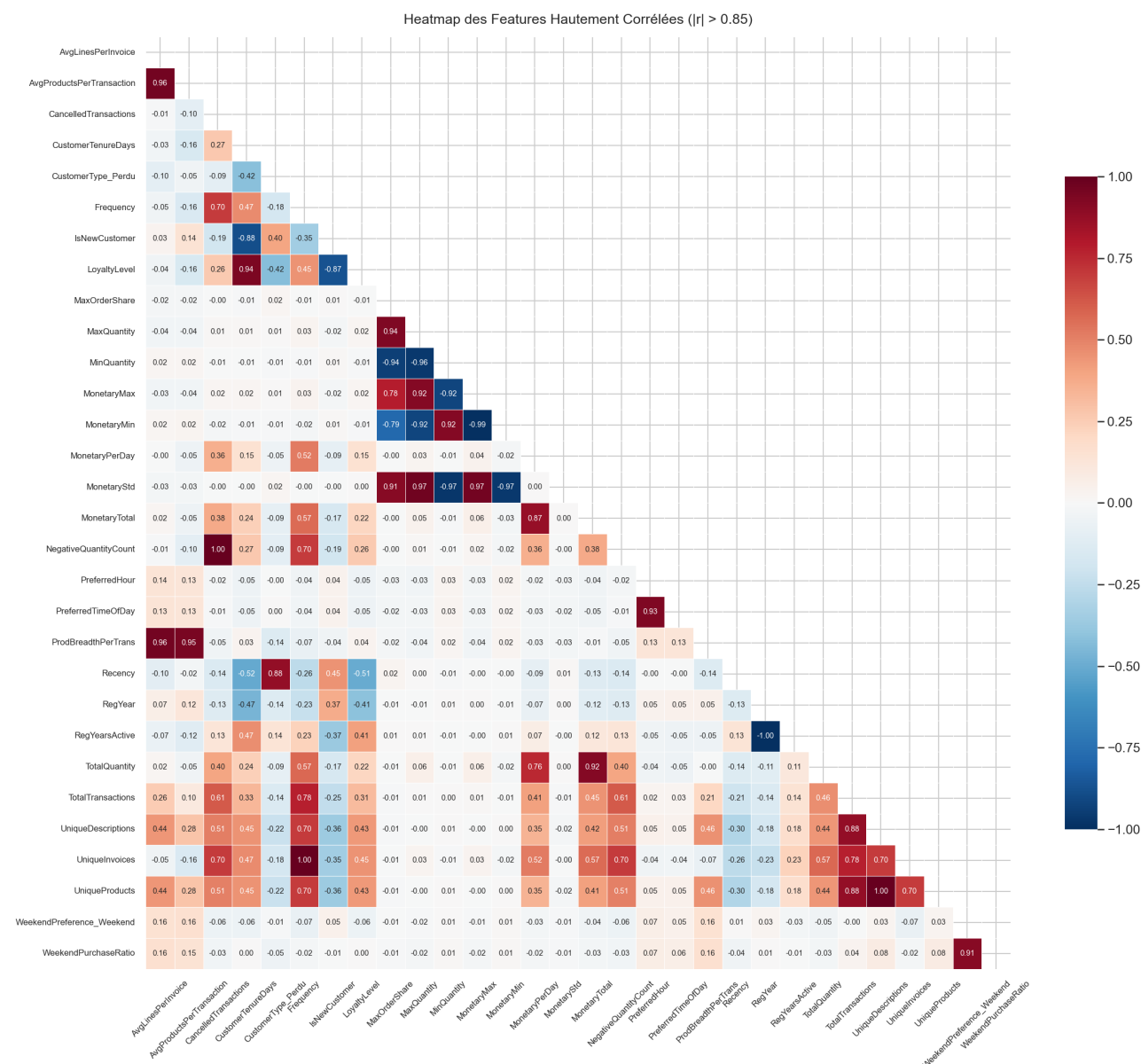


FIGURE 10 – Heatmap des features impliquées dans au moins une paire de corrélation $|r| > 0,85$. Les valeurs annotées permettent d'identifier les redondances exactes ($r = 1,000$).

Feature supprimée	Feature conservée	$ r $
NegativeQuantityCount	CancelledTransactions	1,000
UniqueInvoices	Frequency	1,000
RegYear	RegYearsActive	1,000
UniqueDescriptions	UniqueProducts	1,000
MonetaryMin	MonetaryStd	0,994
MinQuantity	MonetaryStd	0,974
MaxQuantity	MonetaryStd	0,973
AvgLinesPerInvoice	AvgProductsPerTransaction	0,964
AvgProductsPerTransaction	ProdBreadthPerTrans	0,947
LoyaltyLevel	CustomerTenureDays	0,943
MaxOrderShare	MonetaryStd	0,938
WeekendPurchaseRatio	WeekendPreference_Weekend	0,914
IsNewCustomer	CustomerTenureDays	0,884
TotalTransactions	UniqueProducts	0,878
MonetaryPerDay	MonetaryTotal	0,874

Au total, **15 features** sont supprimées à cette étape.

5.6 Critère 4 — Importance nulle (Random Forest)

Un `RandomForestClassifier` (200 arbres, profondeur max 15, `min_samples_leaf=5`) est entraîné sur les 71 features restantes. Les features dont l'importance Gini est strictement nulle sont éliminées.

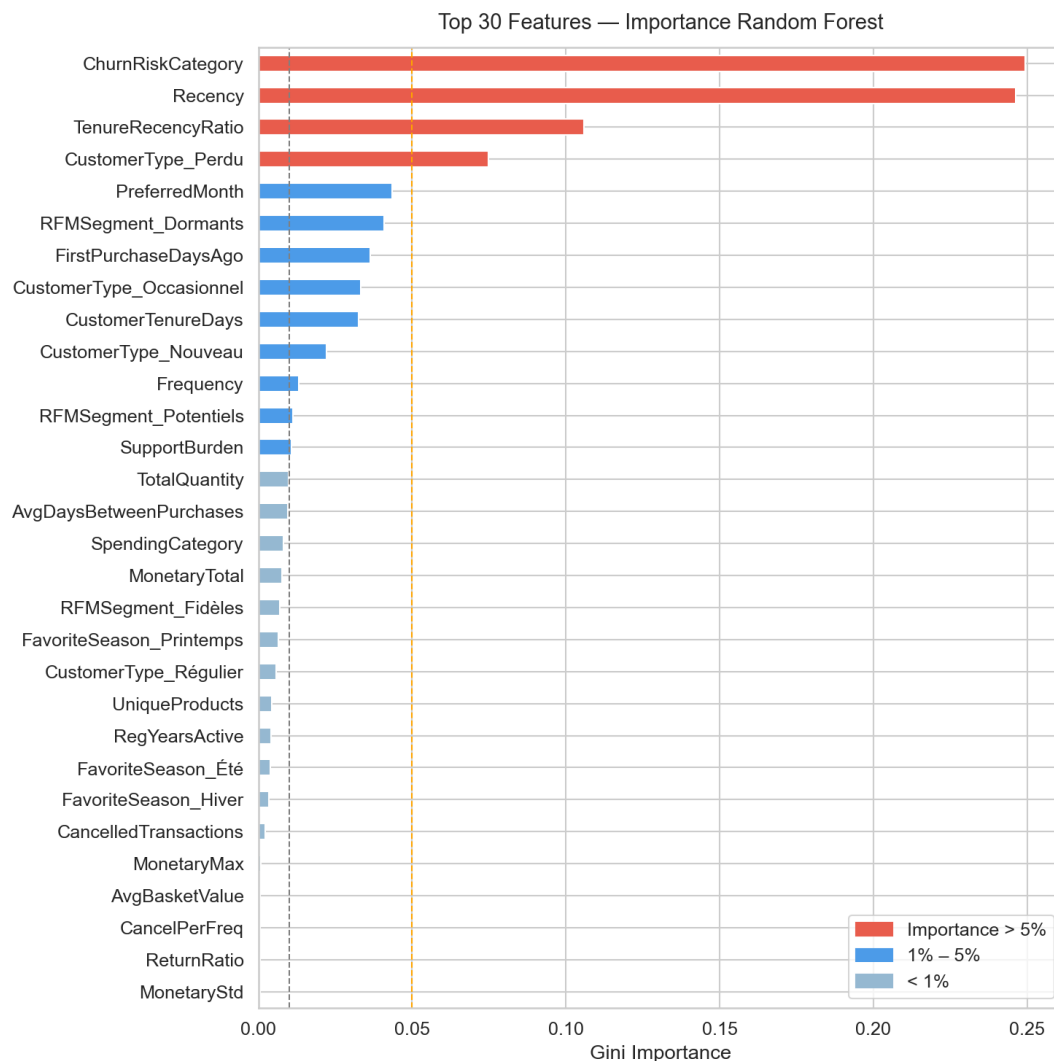


FIGURE 11 – Importance des 30 features les plus informatives selon le Random Forest. Rouge : importance > 5 % ; bleu : 1–5 % ; gris : < 1 %.

Les **13 features à importance Gini nulle** supprimées sont principalement les variables régionales encodées en OHE dont la distribution est quasi-uniforme : UniqueCountries, IP_IsPrivate, Gender_M, Region_Amérique du Nord, AccountStatus_Closed, Region_Asie, Region_Amérique du Sud, Region_Océanie, Region_Moyen-Orient, Region_Europe centrale, Region_Europe de l'Est, Region_Europe du Sud et Region_Europe du Nord.

5.7 Bilan de la sélection

TABLE 4 – Résumé de la sélection de features

Critère	Features éliminées	Résultat cumulé
Features initiales (après encoding)	—	86
Variance nulle	0	86
Taux de NaN > 50 %	0	86
Corrélation inter-features $ r > 0,85$	15	71
Importance RF nulle	13	58
Features conservées		58

6 Étape 4 — Imputation Avancée des Valeurs Manquantes

Règle d'or — Pas de Data Leakage

Le *fit* de l'imputeur se fait **uniquement** sur `X_train`. La même transformation est ensuite appliquée à `X_test` via `transform()`, sans refaire de *fit*. Imputer avant le split constituerait une fuite de données (*data leakage*).

6.1 Split Train/Test préalable

```
1 X_train_raw, X_test_raw, y_train, y_test = train_test_split(
2     X_selected, y, test_size=0.20,
3     random_state=42, stratify=y
4 )
5 # Train : (3497, 58) | Test : (875, 58)
6 # Churn rate -- Train : 0.333 | Test : 0.333
```

6.2 Imputation par médiane

Appliquée aux features avec peu de données manquantes : `AvgDaysBetweenPurchases` (1,89%), `SupportTicketsCount` (3,00%), `SupportBurden` (3,00%) et `IP_FirstOctet`.

6.3 Iterative Imputer — SatisfactionScore

`IterativeImputer` modélise chaque feature avec des NaN comme variable dépendante et prédit les valeurs manquantes par régression sur les autres features. Variables de contexte : `Recency`, `Frequency`, `CustomerTenureDays`, `SpendingCategory`. Valeurs clippées dans [1, 5].

```
1 iter_imputer = IterativeImputer(max_iter=10, random_state=42,
2                                 initial_strategy='median')
3 X_train['SatisfactionScore'] = X_train_iter[:, 0].clip(1, 5)
```

6.4 KNN Imputer — Age (~30% manquant)

`KNNImputer` ($k = 5$) remplace chaque NaN par la moyenne des 5 voisins les plus proches. Features de contexte : `Frequency`, `MonetaryTotal`, `CustomerTenureDays`, `Recency`, `AgeCategory`, `SpendingCategory`. Valeurs clippées dans [18, 81] ans.

```
1 knn_imputer = KNNImputer(n_neighbors=5)
2 knn_result_train = knn_imputer.fit_transform(X_train[knn_context])
3 X_train['Age'] = knn_result_train[:, age_idx].clip(18, 81)
4 # NaN restants apres imputation -- Train : 0 | Test : 0
```

7 Étape 5 — Normalisation (StandardScaler)

Le `StandardScaler` centre les données à moyenne 0 et réduit à écart-type 1. Cette normalisation est indispensable pour les algorithmes sensibles à l'échelle (SVM, KNN, régression logistique, réseaux de neurones) et pour la PCA.

Note importante

Ne **jamais** normaliser la variable cible `y`. Le scaler est fitté **uniquement** sur `X_train` puis appliqué à `X_test`.

7.1 Stratégie sélective

Les 21 colonnes binaires (valeurs $\{0, 1\}$) ont été exclues de la normalisation. Seules les **37 features continues** ont été scalées.

```
1 binary_cols = [c for c in X_train.columns
2                 if X_train[c].nunique() <= 2]
3 cols_to_scale = [c for c in X_train.columns
4                  if c not in binary_cols]
5 scaler = StandardScaler()
6 X_train_scaled[cols_to_scale] = scaler.fit_transform(
7     X_train[cols_to_scale])
8 X_test_scaled[cols_to_scale] = scaler.transform(
9     X_test[cols_to_scale])
10 # Colonnes scalees : 37
11 # Colonnes binaires : 21
```

Le scaler est sauvegardé sous `models/standard_scaler_v2.joblib`.

8 Étape 6 — Analyse en Composantes Principales (ACP)

La PCA est appliquée sur les 58 features standardisées pour explorer la structure de variance des données, identifier le nombre optimal de composantes et créer des représentations alternatives du dataset pour certains modèles.

8.1 Principe et utilité

L'ACP est une méthode de réduction de dimension qui projette les données dans un sous-espace orthogonal en maximisant la variance retenue. Elle est utile pour : la visualisation en 2D/3D du dataset, la réduction du bruit, l'accélération des algorithmes de modélisation, et pour éviter la malédiction de la dimensionnalité.

8.2 Variance expliquée — Scree Plot

La première composante principale (PC1) capte **13,79 %** de la variance totale, la deuxième **7,56 %**, et la décroissance devient progressive à partir de la troisième.

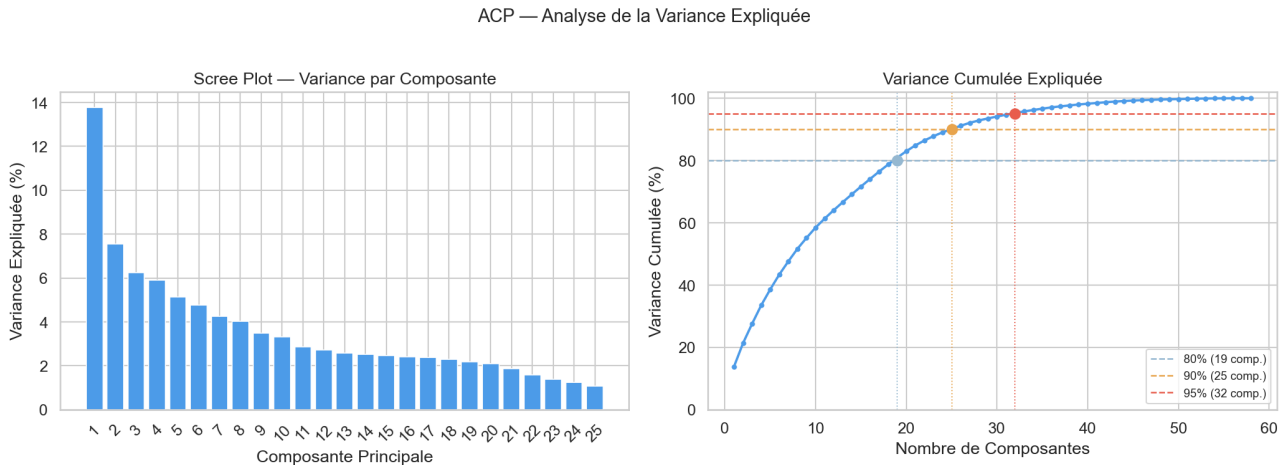


FIGURE 12 – Scree plot (gauche) : variance expliquée par composante. Courbe cumulative (droite) avec seuils 80 %, 90 % et 95 %.

TABLE 5 – Nombre de composantes nécessaires selon le seuil de variance retenue

Seuil	Composantes	Réduction dimensionnelle
80 % de variance	19	67 % (58 → 19)
90 % de variance	25	57 % (58 → 25)
95 % de variance	32	45 % (58 → 32)

La variance expliquée détaillée pour les 15 premières composantes est la suivante :

PC01 : 13,79 %	(cumulé : 13,79 %)
PC02 : 7,56 %	(cumulé : 21,35 %)
PC03 : 6,24 %	(cumulé : 27,59 %)
PC04 : 5,90 %	(cumulé : 33,49 %)
PC05 : 5,14 %	(cumulé : 38,63 %)
PC06 : 4,77 %	(cumulé : 43,40 %)
PC07 : 4,24 %	(cumulé : 47,64 %)
PC08 : 4,02 %	(cumulé : 51,66 %)
PC09 : 3,50 %	(cumulé : 55,16 %)
PC10 : 3,32 %	(cumulé : 58,48 %)
PC11 : 2,87 %	(cumulé : 61,35 %)
PC12 : 2,71 %	(cumulé : 64,06 %)
PC13 : 2,59 %	(cumulé : 66,65 %)
PC14 : 2,52 %	(cumulé : 69,17 %)
PC15 : 2,48 %	(cumulé : 71,65 %)

8.3 Projection 2D des données (PC1 × PC2)

La projection sur les deux premières composantes principales représente **21,35 %** de la variance totale. Le nuage de points ci-dessous colore chaque client selon son statut Churn. On observe une séparation partielle entre les deux classes le long de l'axe PC2 (dominé par **Recency** et **ChurnRiskCategory**), confirmant que ces deux features constituent les dimensions les plus discriminantes pour la prédiction du churn.

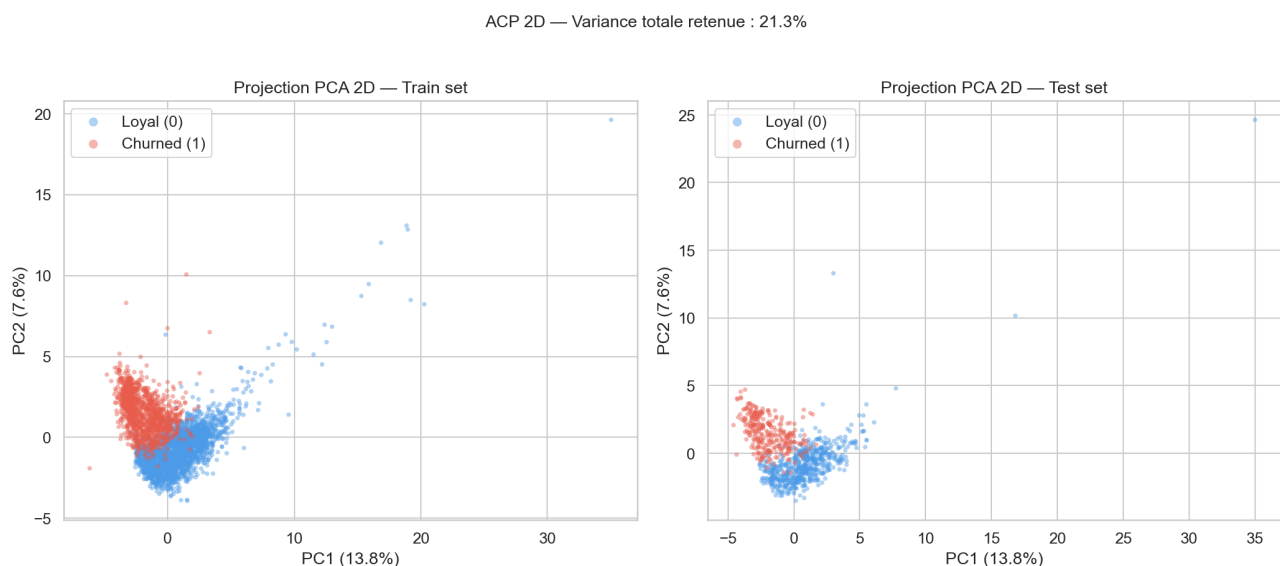


FIGURE 13 – Projection 2D des 4 372 clients sur PC1 × PC2. Train set (gauche) et test set (droite). Les clients churned (rouge) se concentrent dans les valeurs élevées de PC2.

8.4 Biplot — Contribution des features originales

Le biplot superpose les projections des clients et les vecteurs de chargement des features originales sur l'espace PC1/PC2. La longueur d'un vecteur reflète la contribution de la feature à ces deux composantes. **ChurnRiskCategory** et **Recency** tirent fortement dans la direction de PC2, tandis que **CustomerTenureDays** et **SpendingCategory** contribuent davantage à PC1.

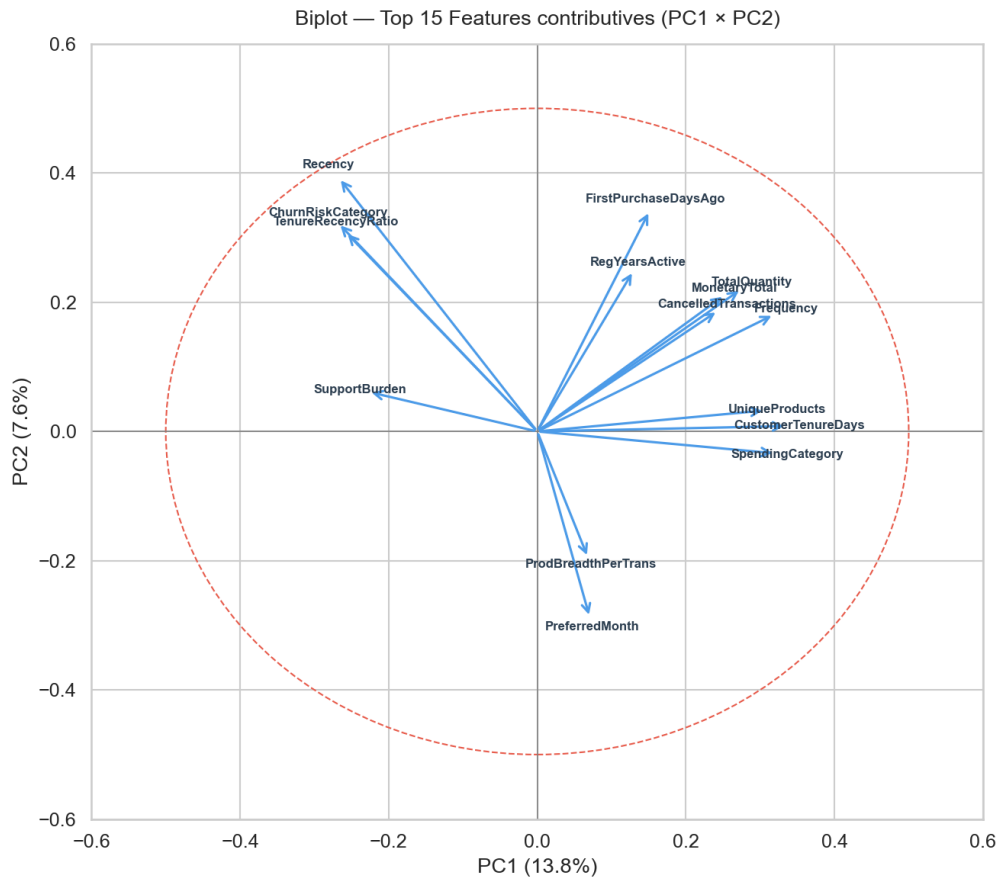


FIGURE 14 – Biplot PCA (PC1 × PC2) : points = clients, flèches = features originales. Seules les 15 features avec la plus grande norme de chargement sont affichées.

8.5 Interprétation des composantes principales

TABLE 6 – Interprétation des 10 premières composantes principales

Comp.	Variance	Features dominantes & interprétation
PC1	13,8 %	CustomerTenureDays, SpendingCategory, Frequency — <i>Profil d'engagement global</i>
PC2	7,6 %	Recency, FirstPurchaseDaysAgo, ChurnRiskCategory — <i>Signal de désengagement récent</i>
PC3	6,2 %	AvgBasketValue, MonetaryAvg, AvgQuantityPerTransaction — <i>Intensité de dépense</i>
PC4	5,9 %	PreferredHour, PreferredTimeOfDay, AvgDaysBetweenPurchases — <i>Habitudes temporelles</i>
PC5	5,1 %	MonetaryStd, MonetaryMax, ReturnRatio — <i>Variabilité des achats</i>
PC6	4,8 %	PreferredHour, PreferredTimeOfDay, MonetaryMax — <i>Comportement d'achat premium</i>
PC7	4,2 %	PreferredTimeOfDay, ProdBreadthPerTrans — <i>Diversité produit temporelle</i>
PC8	4,0 %	AgeCategory, Age, MonetaryAvg — <i>Profil démographique</i>
PC9	3,5 %	RegMonth, SupportTicketsCount, RegYearsActive — <i>Saisonnalité et support</i>
PC10	3,3 %	CancelPerFreq, ProdBreadthPerTrans, RegMonth — <i>Comportement d'annulation</i>

8.6 Trois configurations ACP

Configuration	Composantes	Variance retenue	Usage
PCA 2D	2	21,35%	Visualisation
PCA-10	10	58,48%	Modélisation ML
PCA-90%	25	90,14%	Conservation maximale

9 Étape 7 — Rééquilibrage des Classes (SMOTE)

Le dataset présente un déséquilibre de classes (ratio $\approx 2 : 1$). SMOTE (*Synthetic Minority Over-sampling Technique*) génère des exemples synthétiques de la classe minoritaire par interpolation entre k voisins proches existants.

```

1 smote = SMOTE(random_state=42, k_neighbors=5)
2 X_train_smote, y_train_smote = smote.fit_resample(
3     X_train_scaled, y_train)
4 # Distribution Churn avant SMOTE : 0 -> 2334, 1 -> 1163
5 # Distribution Churn après SMOTE : 0 -> 2334, 1 -> 2334

```

Classe	Avant SMOTE	Après SMOTE
Churn = 0 (Loyal)	2 334	2 334
Churn = 1 (Parti)	1 163	2 334
Total	3 497	4 668

Important

Le SMOTE est appliqué **uniquement sur X_train**. Ne jamais appliquer SMOTE sur X_test, afin de conserver une évaluation fidèle à la distribution réelle.

10 Étape 8 — Export des Datasets et Modèles

10.1 Datasets exportés

Fichier	Dimensions	Usage
X_train_scaled.csv	$3\,497 \times 58$	Modèles classiques
X_test_scaled.csv	875×58	Évaluation
X_train_scaled_smote.csv	$4\,668 \times 58$	Avec rééquilibrage
X_train_pca10.csv	$3\,497 \times 10$	Modèles PCA
X_test_pca10.csv	875×10	Évaluation PCA
X_train_pca10_smote.csv	$4\,668 \times 10$	PCA + SMOTE
X_train_pca90.csv	$3\,497 \times 25$	PCA 90% variance
X_test_pca90.csv	875×25	Évaluation PCA 90%

10.2 Modèles sauvegardés

- `standard_scaler_v2.joblib` — Scaler pour features continues
- `knn_imputer_age.joblib` — KNN Imputer pour Age
- `iterative_imputer_satisfaction.joblib` — Iterative Imputer pour SatisfactionScore
- `pca_2d.joblib`, `pca_10.joblib`, `pca_90.joblib` — Trois modèles PCA

11 Conclusion — Datasets prêts pour la modélisation

À l'issue de ce pipeline de préparation, trois représentations du dataset sont disponibles pour la modélisation, chacune adaptée à un type de modèle différent :

TABLE 7 – Récapitulatif des datasets disponibles pour la modélisation

Dataset	Train	Test	Usage recommandé
X_selected (58 features)	$3\,497 \times 58$	875×58	Modèles interprétables (LR, RF, XGBoost)
X_pca10 (10 composantes)	$3\,497 \times 10$	875×10	Modèles rapides, visualisation
X_pca25 (25 comp., 90 %)	$3\,497 \times 25$	875×25	Compromis performance / vitesse

Ce notebook constitue une étape clé de la chaîne de data science appliquée à la prédiction du churn client. Les principales réalisations sont :

1. Un **pipeline de prétraitement complet et reproductible**, avec séparation stricte train/test pour éviter tout data leakage.
2. Une **sélection de features rigoureuse** combinant critères statistiques (variance, corrélation) et critères modèle (importance Random Forest), réduisant le dataset de 86 à 58 features.
3. Une **stratégie d'imputation adaptée** à chaque feature selon sa distribution et son taux de valeurs manquantes.
4. Trois **configurations ACP** pour répondre à différents besoins : visualisation (2D), modélisation (10 composantes), conservation (90% variance).
5. Un **rééquilibrage par SMOTE** pour compenser le déséquilibre des classes.

Les datasets SMOTE (équilibrés, 4 668 lignes) sont utilisés pour l'entraînement, tandis que les ensembles de test conservent la distribution naturelle (33,3 % de churn) pour une évaluation fidèle à la réalité. Le pipeline est entièrement reproductible et sans fuite de données entre les partitions.

Prêt pour la modélisation

Clustering · Classification · Régression · Déploiement Flask